Problem 1: Social Media Username Availability Checker

Scenario: You're building a registration system for a social media platform with 10 million users. Users frequently check if usernames are available before registering.

Problem Statement: Design a system to check username availability in real-time. The system should:

- Check if a username exists in O(1) time
- Handle 1000 concurrent username checks per second
- Suggest similar available usernames if the requested one is taken
- Track popularity of attempted usernames

Concepts Covered:

- Hash table basics (key-value mapping)
- O(1) lookup performance
- Collision handling
- Frequency counting

Hints:

```
// Use HashMap for instant lookup
// Store username -> userId mapping
// Use separate HashMap for attempt frequency
// For suggestions: append numbers or modify characters
```

Use Cases:
- Twitter/Instagram registration
- Gaming platform username selection
- Email address availability checking

Sample Input/Output:
```
checkAvailability("john_doe") → false (already taken)
checkAvailability("jane_smith") → true (available)
suggestAlternatives("john_doe") → ["john_doe1", "john_doe2", "john.doe"]
getMostAttempted() → "admin" (10,543 attempts)
```

Problem 2: E-commerce Flash Sale Inventory Manager

Scenario: During a flash sale, 50,000 customers simultaneously try to purchase limited stock items (only 100 units available). You need to prevent overselling while maintaining high performance.

Problem Statement: Implement an inventory management system that:

- Tracks product stock levels in real-time
- Processes purchase requests in O(1) time
- Handles concurrent requests safely
- Maintains a waiting list when stock runs out
- Provides instant stock availability checks

Concepts Covered:

- Hash table for instant stock lookup
- Collision resolution (multiple users buying same product)
- Load factor management during high traffic
- Performance benchmarking under load

Hints:

```
// HashMap<productId, stockCount>
// Synchronize decrement operations
// Use LinkedHashMap for waiting list (FIFO)
// Implement atomic operations for thread safety
```

Use Cases:
- Amazon Prime Day deals
- Concert ticket booking systems
- Limited edition product launches

Sample Input/Output:
```
checkStock("IPHONE15_256GB") → 100 units available
purchaseItem("IPHONE15_256GB", userId=12345) → Success, 99 units remaining
purchaseItem("IPHONE15_256GB", userId=67890) → Success, 98 units remaining
... (after 100 purchases)
purchaseItem("IPHONE15_256GB", userId=99999) → Added to waiting list, position #1
```

Problem 3: DNS Cache with TTL (Time To Live)

Scenario: Build a DNS resolver cache that stores domain-to-IP mappings to reduce lookup times from 100ms to <1ms. Cache entries should expire after a specified TTL.

Problem Statement: Create a DNS caching system that:

- Stores domain name → IP address mappings
- Implements TTL-based expiration (entries expire after X seconds)
- Automatically removes expired entries
- Handles cache misses by querying upstream DNS
- Reports cache hit/miss ratios
- Implements LRU eviction when cache is full

Concepts Covered:

- Hash table implementation with custom Entry class
- Chaining for collision resolution
- Time-based operations
- Performance metrics

Hints:

```
// Entry class: domain, ipAddress, timestamp, expiryTime
// HashMap<String, DNSEntry>
// Background thread to clean expired entries
// Track hits/misses for statistics
```

Use Cases:
- Browser DNS caching
- CDN edge server DNS resolution
- Corporate network DNS servers

Sample Input/Output:
```
resolve("google.com") → Cache MISS → Query upstream → 172.217.14.206 (TTL: 300s)
resolve("google.com") → Cache HIT → 172.217.14.206 (retrieved in 0.2ms)
... after 301 seconds ...
resolve("google.com") → Cache EXPIRED → Query upstream → 172.217.14.207
getCacheStats() → Hit Rate: 87.5%, Avg Lookup Time: 0.8ms
```

Problem 4: Plagiarism Detection System

Scenario: A university needs to check student submissions against a database of 100,000 previous essays to detect plagiarism. Simple string matching is too slow.

Problem Statement: Build a plagiarism detector that:

- Breaks documents into n-grams (sequences of n words)
- Stores n-grams in a hash table with document references
- Finds matching n-grams between documents
- Calculates similarity percentage
- Identifies the most similar documents in O(n) time

Concepts Covered:

- String hashing techniques
- Frequency counting with hash maps
- Good hash function properties
- Performance benchmarking (hash vs. linear search)

Hints:

```
// HashMap<String, Set<DocumentId>> for n-gram → documents mapping
// Use 5-grams or 7-grams for better accuracy
// Compute hash for each n-gram window
// Count matching n-grams to calculate similarity
```

Use Cases:
- Academic plagiarism detection (Turnitin)
- Code similarity detection (MOSS)
- Document deduplication systems

Sample Input/Output:
```
analyzeDocument("essay_123.txt")
→ Extracted 450 n-grams
→ Found 67 matching n-grams with "essay_089.txt"
→ Similarity: 14.9% (suspicious)
→ Found 312 matching n-grams with "essay_092.txt"
→ Similarity: 69.3% (PLAGIARISM DETECTED)
```

Problem 5: Real-Time Analytics Dashboard for Website Traffic

Scenario: A news website gets 1 million page views per hour. The marketing team needs real-time analytics showing top pages, traffic sources, and user locations.

Problem Statement: Implement a streaming analytics system that:

- Processes incoming page view events in real-time
- Maintains top 10 most visited pages
- Tracks unique visitors per page
- Counts visits by traffic source (Google, Facebook, Direct, etc.)
- Updates dashboard every 5 seconds with zero lag

Concepts Covered:

- Frequency counting applications
- Multiple hash tables for different dimensions
- Load factor and resizing under high throughput
- Time/space complexity optimization

Hints:

```
// HashMap<pageUrl, visitCount> for page views
// HashMap<pageUrl, Set<userId>> for unique visitors
// HashMap<source, count> for traffic sources
// Use LinkedHashMap or PriorityQueue for top N
// Batch updates every 5 seconds
```

Use Cases:
- Google Analytics real-time dashboard
- Twitter trending topics
- E-commerce product popularity tracking

Sample Input/Output:
```
processEvent({url: "/article/breaking-news", userId: "user_123", source: "google"})
processEvent({url: "/article/breaking-news", userId: "user_456", source: "facebook"})
...
getDashboard() →
  Top Pages:
    1. /article/breaking-news - 15,423 views (8,234 unique)
    2. /sports/championship - 12,091 views (9,871 unique)
```

Traffic Sources:
Google: 45%, Direct: 30%, Facebook: 15%, Other: 10%

Problem 6: Distributed Rate Limiter for API Gateway

Scenario: Your API gateway handles requests from 100,000 clients. Each client is allowed 1000 requests per hour. You need to enforce this limit efficiently.

Problem Statement: Build a token bucket rate limiter that:

- Tracks request counts per client (by API key or IP)
- Allows burst traffic up to limit
- Resets counters every hour
- Responds within 1ms for rate limit checks
- Handles distributed deployment (multiple servers)
- Provides clear error messages when limit exceeded

Concepts Covered:

- Hash table for client tracking
- Time-based operations
- Collision handling (multiple clients)
- Performance under concurrent access

Hints:
```
// HashMap<clientId, TokenBucket>

// TokenBucket: {tokens, lastRefillTime, maxTokens, refillRate}
// Atomic operations for thread safety
// Sliding window or fixed window algorithm
```

Use Cases:
- AWS API Gateway rate limiting
- GitHub API rate limits
- Stripe payment API throttling

Sample Input/Output:
```
checkRateLimit(clientId="abc123") → Allowed (998 requests remaining)
checkRateLimit(clientId="abc123") → Allowed (997 requests remaining)...
checkRateLimit(clientId="abc123") → Denied (0 requests remaining, retry after 3540s)
getRateLimitStatus("abc123") → {used: 1000, limit: 1000, reset: 1675890000}
```

Problem 7: Autocomplete System for Search Engine

Scenario: Build a Google-like autocomplete that suggests queries as users type, based on 10 million previous search queries and their popularity.

Problem Statement: Create an autocomplete system that:

- Stores search queries with frequency counts
- Returns top 10 suggestions for any prefix in <50ms
- Updates frequencies based on new searches
- Handles typos and suggests corrections
- Optimizes for memory (10M queries × avg 30 characters)

Concepts Covered:

- Hash table for query frequency storage
- String hashing techniques
- Performance benchmarking (prefix search)
- Space complexity optimization

Hints:

```
// HashMap<query, frequency> for global stats
// Trie + HashMap hybrid for prefix matching
// Cache popular prefix results
// Use min-heap for top K results
```

Use Cases:
- Google search autocomplete
- Amazon product search suggestions
- IDE code completion

Sample Input/Output:
```
search("jav") →
  1. " tutorial" (1,234,567 searches)
  2. "script" (987,654 searches)
  3. " download" (456,789 searches)
  ...

updateFrequency(" 21 features") → Frequency: 1 → 2 → 3 (trending)
```

Problem 8: Parking Lot Management with Open Addressing

Scenario: A smart parking lot with 500 spots needs to track which vehicles are parked where, handle collisions when multiple vehicles arrive simultaneously, and optimize spot allocation.

Problem Statement: Implement a parking system using open addressing that:

- Assigns parking spots based on license plate hash
- Uses linear probing when preferred spot is occupied
- Tracks entry/exit times for billing
- Finds nearest available spot to entrance
- Generates parking statistics (avg occupancy, peak hours)

Concepts Covered:

- Open addressing (linear/quadratic probing)
- Collision resolution strategies
- Custom hash functions
- Load factor management

Hints:

```
// Array-based hash table with open addressing
// Hash function: licensePlate → preferred spot number
// Linear probing: spot, spot+1, spot+2, ...
// Track spot status: EMPTY, OCCUPIED, DELETED
```

Use Cases:
- Airport parking systems
- Mall parking management
- Street parking apps (ParkMobile)

Sample Input/Output:
```
parkVehicle("ABC-1234") → Assigned spot #127 (0 probes)
parkVehicle("ABC-1235") → Assigned spot #127... occupied... Spot #128 (1 probe)
parkVehicle("XYZ-9999") → Assigned spot #127... occupied... occupied... Spot #129 (2 probes)
exitVehicle("ABC-1234") → Spot #127 freed, Duration: 2h 15m, Fee: $12.50
getStatistics() → Occupancy: 78%, Avg Probes: 1.3, Peak Hour: 2-3 PM
```

Problem 9: Two-Sum Problem Variants for Financial Transactions

Scenario: A payment processing company needs to detect fraudulent transaction pairs that sum to specific amounts (money laundering), find complementary trades, and identify duplicate payments.

Problem Statement: Given millions of daily transactions, implement:

- Classic Two-Sum: Find pairs that sum to target amount
- Two-Sum with time window: Pairs within 1 hour
- K-Sum: Find K transactions that sum to target
- Duplicate detection: Same amount, same merchant, different accounts
- All under 100ms response time

Concepts Covered:

- Hash table for complement lookup
- O(1) lookup performance
- Multiple hash tables for different checks
- Time complexity analysis

Hints:

```
// HashMap<complement, transaction> for two-sum
// HashMap<amount, List<Transaction>> for duplicates
// For time window: filter by timestamp first
// K-Sum: recursive with hash table memoization
```

Use Cases:
- Fraud detection systems
- Tax evasion detection
- Cryptocurrency transaction analysis

Sample Input/Output:

```
transactions = [
  {id:1, amount:500, merchant:"Store A", time:"10:00"},
  {id:2, amount:300, merchant:"Store B", time:"10:15"},
  {id:3, amount:200, merchant:"Store C", time:"10:30"},
]
findTwoSum(target=500) → [(id:2, id:3)] // 300 + 200
detectDuplicates() → [{amount:500, merchant:"Store A", accounts:[acc1, acc2]}]
findKSum(k=3, target=1000) → [(id:1, id:2, id:3)] // 500+300+200
```

Problem 10: Multi-Level Cache System with Hash Tables

Scenario: Design a cache hierarchy for a video streaming service (like Netflix) with L1 (memory), L2 (SSD), and L3 (database) levels. Optimize for 10M concurrent users.

Problem Statement: Build a multi-level caching system that:

- L1 Cache: 10,000 most popular videos (in-memory HashMap)
- L2 Cache: 100,000 frequently accessed videos (SSD-backed)
- L3: Database (slow, all videos)
- Implements LRU eviction at each level
- Promotes videos between levels based on access patterns
- Tracks cache hit ratios for each level
- Handles cache invalidation when content updates

Concepts Covered:

- Multiple hash tables with different purposes
- Resizing/rehashing strategies
- Performance benchmarking across levels
- Load factor optimization for each tier

Hints:

```
// L1: LinkedHashMap<videoId, VideoData> (access-order)
// L2: HashMap<videoId, FilePath> (points to SSD location)
// Track access count: HashMap<videoId, accessCount>
// Promote video from L2→L1 if access count > threshold
// Implement LRU with LinkedHashMap or custom doubly-linked list
```

Use Cases:
- Netflix video streaming cache
- CDN content delivery
- Database query result caching

Sample Input/Output:
```
getVideo("video_123")
→ L1 Cache MISS (0.5ms)
→ L2 Cache HIT (5ms)
→ Promoted to L1
```

→ Total: 5.5ms

getVideo("video_123") [second request]
→ L1 Cache HIT (0.5ms)

getVideo("video_999")
→ L1 Cache MISS
→ L2 Cache MISS
→ L3 Database HIT (150ms)
→ Added to L2 (access count: 1)

getStatistics() →
L1: Hit Rate 85%, Avg Time: 0.5ms
L2: Hit Rate 12%, Avg Time: 5ms
L3: Hit Rate 3%, Avg Time: 150ms
Overall: Hit Rate 97%, Avg Time: 2.3ms